

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 8

Kernel System Calls, Userland Process Orchestration, and Pipe-Based Communication in the xv6 Operating System

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL XV6 KERNEL SYSTEM CALL AND IPC ARCHITECTURE **3**

1.1 RISC-V user-to-supervisor privilege transitions and trap dispatching 4

1.2 Process address space management and lifecycle in xv6 4

1.3 xv6 pipe buffer mechanics and synchronization primitives 5

1.4 Theoretical complexity comparison 6

2 SYSTEM IMPLEMENTATION AND EXPERIMENTAL VALIDATION **6**

2.1 Process orchestration inside xv6 (user/run.c) 7

2.2 Bidirectional inter-process communication via pingpong.c 8

2.3 Concurrent prime number sieve pipeline (user/primes.c) 9

CONCLUSIONS **10**

REFERENCES **11**

ANNEX A **12**

ANNEX B **14**

INTRODUCTION

The core abstraction of an operating system kernel is the provision of a secure, isolated boundary between userland application programs and privileged hardware resources. In contemporary monolithic operating systems, this interface is realized through system calls. System calls permit unprivileged user code running in CPU user mode to request services from the kernel executing in privileged supervisor mode. While modern industrial kernels like Linux comprise millions of lines of source code and hundreds of specialized system call primitives, the fundamental design principles governing process control, address space replacement, and inter-process communication remain rooted in the classic Unix paradigm.

The educational xv6 operating system, developed at the Massachusetts Institute of Technology, provides a clean, modern re-implementation of Unix Sixth Edition targeting the 64-bit RISC-V processor architecture under QEMU emulation. Stripped of industrial complexity, xv6 implements the exact same foundational system calls studied on Linux—namely `fork()`, `exec()`, `wait()`, `pipe()`, `read()`, and `write()`—directly on top of raw hardware structures, page tables, and hardware traps. Investigating how these system calls operate within xv6 provides direct insight into how the kernel and userland cooperate without the mediating layers of modern C libraries.

The objective of this laboratory work is to implement userland process management and inter-process communication programs within the xv6 operating system. The assignment entails implementing a userland program (`user/run.c`) that executes the canonical three-step shell lifecycle (`fork`, `exec`, `wait`) natively inside xv6, analyzing differences in status reporting compared to Linux, constructing a bidirectional inter-process communication utility (`user/pingpong.c`) utilizing two unidirectional pipes, and developing a concurrent prime number sieve pipeline (`user/primes.c`). All source modifications, build configurations, and verification routines are maintained in the version control repository hosted on GitHub at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL XV6 KERNEL SYSTEM CALL AND IPC ARCHITECTURE

1.1 RISC-V user-to-supervisor privilege transitions and trap dispatching

In the RISC-V architecture, processors support multiple privilege tiers: User mode (U-mode), Supervisor mode (S-mode), and Machine mode (M-mode). Application programs execute strictly in unprivileged U-mode, where direct manipulation of control registers, memory management page tables, and I/O devices is physically prevented by hardware logic [1], [2].

When a user program issues a system call, it prepares the system call number in register `a7`, places parameters in registers `a0` through `a5`, and executes the `ecall` instruction. The `ecall` instruction triggers an environment call exception, transferring execution from U-mode to S-mode. Hardware saves the current program counter into the Supervisor Exception Program Counter register (`sepc`), stores the trap cause in `scause`, and jumps to the address stored in the Supervisor Trap Vector Base Address register (`stvec`).

In xv6, `stvec` points to the trampoline page, a single page mapped at the very top of virtual memory (`0x3FFFFFF000`) with identical virtual addresses across both user and kernel address spaces. The trampoline assembly code saves all 32 general-purpose user registers into the process trapframe structure (`struct trapframe`), switches the page table pointer register (`satp`) from the user page table to the kernel page table, flushes the TLB, and calls the C kernel function `usertrap()` located in `kernel/trap.c`.

Inside `usertrap()`, the kernel checks whether `scause` indicates an environment call (cause $==$ 8). If so, it increments `p->trapframe->epc` by 4 bytes to ensure the process resumes at the subsequent instruction after trap return, enables supervisor interrupts, and invokes `syscall()` in `kernel/syscall.c`. The `syscall()` routine reads register `a7` from the trapframe:

$$\text{num} = p\text{->trapframe->a7} \tag{1.1}$$

It indexes into a static function pointer array `syscalls[num]`, retrieving arguments via helper functions (`argint()`, `argaddr()`, `argstr()`) that read directly from the saved trapframe registers. Upon completion, the return value is written into `p->trapframe->a0`, and `usertrapret()` restores the user registers and executes `sret` to return to user mode.

1.2 Process address space management and lifecycle in xv6

Every process in xv6 is represented by a process descriptor (`struct proc`) stored in a static array of 64 entries (`NPROC = 64`) in `kernel/proc.c`. A process transitions through five primary lifecycle states: `UNUSED`, `USED`, `SLEEPING`, `RUNNABLE`, `RUNNING`, and `ZOMBIE`.

The RISC-V 64-bit architecture implements the Sv39 virtual memory translation scheme. The 39-bit virtual address is segmented into three 9-bit Virtual Page Numbers ($\text{VPN}_2, \text{VPN}_1, \text{VPN}_0$) and a 12-bit page offset ($P_{\text{offset}} = 4096$ bytes):

$$\text{VA} = (\text{VPN}_2 \ll 30) \vee (\text{VPN}_1 \ll 21) \vee (\text{VPN}_0 \ll 12) \vee P_{\text{offset}} \tag{1.2}$$

Translation traverses a 3-level radix tree where each page directory contains 512 64-bit Page Table Entries (PTEs).

When `fork()` is executed, the kernel allocates a new process descriptor via `allocproc()` and duplicates the entire user address space using `uvmcopy()`. Unlike Linux, which employs Copy-on-Write page sharing, base xv6 allocates physical memory frames for every user page and copies the 4096-byte memory content directly. The parent copies all file descriptor pointers, incrementing their reference counts (`f->ref++`), sets the child return value in `trapframe->a0 = 0`, and transitions the child state to `RUNNABLE`.

The `exec(path, argv)` system call replaces the calling process memory image. The kernel opens the binary file from disk, parses the ELF header, verifies the ELF magic constant (`ELF_MAGIC = 0x464C457F`), allocates an entirely new user page table via `proc_pagetable()`, and maps each ELF program segment into memory using `loadseg()`. It allocates two pages at the top of the user address space: an unmapped guard page to catch stack overflows, followed by the user stack page. It pushes the argument strings and argument pointer array onto the stack, frees the old page table via `proc_freepagetable()`, installs the new page table in `p->pagetable`, and sets `p->trapframe->epc` to the ELF entry point.

1.3 xv6 pipe buffer mechanics and synchronization primitives

Inter-process communication in xv6 is provided by anonymous pipes implemented in `kernel/pipe.c`. A pipe is represented by `struct pipe`:

```
struct pipe {
  struct spinlock lock;
  char data[PIPESIZE];
  uint nread;     // number of bytes read
  uint nwrite;    // number of bytes written
  int readopen;   // read fd is still open
  int writeopen;  // write fd is still open
};
```

where *PIPESIZE* is fixed at 512 bytes.

The pipe operates as a circular FIFO buffer. The array index for reading and writing is derived arithmetically using modulo arithmetic over the continuous monotonic byte counters:

$$I_{\text{read}} = n_{\text{read}} \bmod PIPESIZE, \quad I_{\text{write}} = n_{\text{write}} \bmod PIPESIZE \tag{1.3}$$

The instantaneous volume of unread buffered data V_{buffered} is given by:

$$V_{\text{buffered}} = n_{\text{write}} - n_{\text{read}} \tag{1.4}$$

When a process executes `piperead()` and $V_{\text{buffered}} == 0$, the calling process releases the pipe spinlock and transitions to sleep on the channel `&p->nread` via `sleep()`. When a writer executes `pipewrite()`, it copies bytes into `p->data`, increments `nwrite`, and calls `wakeup(&p->nread)`, waking all sleeping readers. If $V_{\text{buffered}} == PIPESIZE$, the writer sleeps on `&p->nwrite` until space is freed.

When all write descriptors referencing a pipe are closed, the kernel sets `p->writeopen = 0` and wakes any sleeping readers. Upon waking, the reader observes that $V_{\text{buffered}} == 0$ and writeopen $== 0$, immediately returning 0 to signal End-of-File (EOF).

1.4 Theoretical complexity comparison

The operational complexity of xv6 system calls and inter-process communication mechanisms is determined by kernel table indexing, memory allocation, and context switching overhead. Table 1.1 summarizes the asymptotic time and space complexities across fundamental xv6 primitives [3], [4].

Table 1.1 – Asymptotic time and space complexities of xv6 system calls and IPC primitives

xv6 Operation	**Time (Average)**	**Time (Worst)**	**Space Complexity**
RISC-V Trap Dispatch (`ecall`)	$O(1)$	$O(1)$	$O(1)$
Process Fork (`fork`)	$O(M_{\text{user}})$	$O(M_{\text{user}})$	$O(M_{\text{user}})$
ELF Image Replacement (`exec`)	$O(S_{\text{ELF}})$	$O(S_{\text{ELF}})$	$O(S_{\text{ELF}})$
Process Wait (`wait`)	$O(N_{\text{proc}})$	$O(N_{\text{proc}})$	$O(1)$
Pipe Byte Transfer (`pipe`)	$O(B_{\text{trans}})$	$O(B_{\text{trans}})$	$O(1)$
Concurrent Prime Sieve Pipeline	$O(N \cdot P)$	$O(N \cdot P)$	$O(P \cdot PIPESIZE)$

In Table 1.1, parameter M_{user} denotes the total physical memory volume allocated to the user address space copied during `fork()`, S_{ELF} represents the executable binary size loaded from the disk image, N_{proc} denotes the static process table size ($N_{\text{proc}} = 64$), B_{trans} represents the byte count transferred across the 512-byte pipe buffer, N denotes the upper bound integer evaluated by the prime sieve ($N = 35$), and P denotes the count of generated prime pipeline stages ($P = 11$).

2 SYSTEM IMPLEMENTATION AND EXPERIMENTAL VALIDATION

2.1 Process orchestration inside xv6 (user/run.c)

The initial practical phase evaluated process lifecycle mechanics inside xv6 through the implementation of `user/run.c`. In xv6, standard userland programs interact directly with the kernel system calls declared in `user/user.h`. The `run` utility duplicates the invoking process via `fork()`. In the child branch (pid $==$ 0), the process outputs its process identifier using `getpid()` and transitions to `echo` via `exec("echo", args)`. The parent process invokes `wait(&status)` to synchronize with child termination.

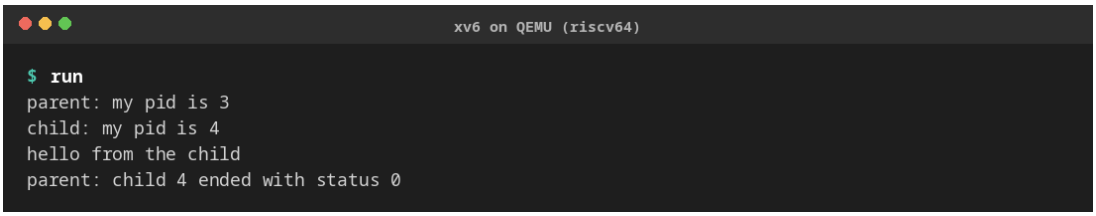


Figure 2.1 – Process creation and execution workflow with run inside xv6

As shown in Figure 2.1, executing `run` inside the xv6 shell yielded deterministic process lifecycle behavior. The parent process reported PID 3, while the newly created child process was assigned PID 4. The child successfully executed `exec("echo", args)`, which completely replaced the address space of `run` with the binary image of `echo`. The message `hello from the child` was printed directly by `echo`, rather than `run`. Upon termination of `echo`, the parent harvested the child exit code, reporting that child 4 ended with status 0.

To evaluate program replacement with varying argument lists and failure conditions, `run.c` was enhanced to accept userland command arguments, supporting execution of `run ls` and `run nosuchprog`.



Figure 2.2 – Program replacement and execution error handling in xv6

According to Figure 2.2, executing `run ls` successfully replaced the child process (PID 6) with

`ls`, emitting the root directory table listing and terminating with status 0. When `run nosuchprog` was invoked, the `exec()` system call failed because the binary `nosuchprog` does not exist in the xv6 root directory. Because `exec()` failed, execution resumed at the subsequent instruction in `run.c`, which printed `exec failed` and called `exit(1)`.

Crucially, this experiment demonstrates a fundamental distinction between xv6 and Linux:

a) On Linux, `run.c` explicitly invoked `exit(127)` following POSIX standard convention where 127 signifies command not found, which the parent extracted via `WEXITSTATUS(status)`;

b) In xv6, `user/run.c` explicitly invoked `exit(1)`, causing the parent to record raw exit status 1.

2.2 Bidirectional inter-process communication via pingpong.c

The second experimental phase examined inter-process communication across concurrent processes through the construction of `user/pingpong.c`. In this design, a parent process and child process exchange a single byte across two dedicated anonymous pipes: $p2c$ (Parent-to-Child) and $c2p$ (Child-to-Parent).

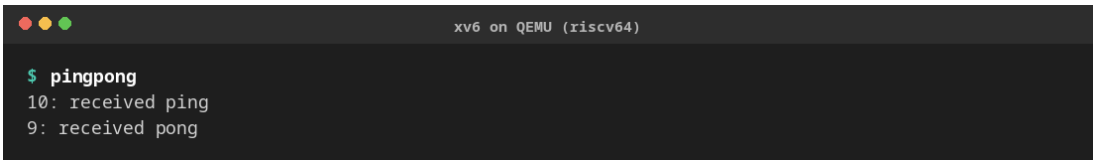


Figure 2.3 – Bidirectional pipe communication (pingpong) in xv6

Figure 2.3 demonstrates the execution of `pingpong`. The child process (PID 10) printed `10: received ping`, while the parent process (PID 9) subsequently printed `9: received pong`.

This implementation illustrates two fundamental operating system principles:

a) **Need for Two Pipes:** Because a Unix pipe is strictly a unidirectional byte queue, attempting bidirectional exchange over a single pipe introduces race conditions. A process writing a byte could immediately read back its own byte before the peer process is scheduled, leading to corrupted exchanges or deadlock. Allocating separate $p2c$ and $c2p$ channels guarantees isolated, unidirectional data flow;

b) **Implicit Blocking Synchronization:** Although the parent contains no explicit synchronization statement between writing and reading, the system call `read(c2p[0], &b, 1)` forces the parent to wait. Because the child has not yet emitted the response byte into $c2p[1]$, the kernel transitions the parent process into the `SLEEPING` state. The parent is resumed only when the child executes `write(c2p[1], &b, 1)`, which triggers `wakeup()` in the kernel pipe driver.

2.3 Concurrent prime number sieve pipeline (user/primes.c)

The optional extension evaluated multi-process pipeline concurrency through the implementation of Doug McIlroy's prime sieve algorithm in `user/primes.c`. The main process generates integers $2 \leq n \leq 35$ into an initial pipe and forks the first sieve stage.



Figure 2.4 – Concurrent prime sieve pipeline execution in xv6

As shown in Figure 2.4, executing `primes` generated all eleven prime numbers between 2 and 35 in strictly increasing order (`2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31`).

Each pipeline stage operates autonomously:

a) It reads the first integer p from its upstream pipe, which is mathematically guaranteed to be prime, and prints `prime p`;

b) It creates a new downstream pipe via `pipe()` and forks a child process for the subsequent prime stage;

c) It iterates over incoming numbers from upstream, filtering out all multiples ($n \bmod p == 0$) and writing coprime numbers ($n \bmod p \neq 0$) into the downstream pipe;

d) Upon reaching EOF, it closes its pipe endpoints, calls `wait(0)` to reap its child, and exits cleanly.

Because xv6 imposes a strict per-process limit of 16 open file descriptors (`NOFILE = 16`), each stage systematically closes unneeded pipe descriptors immediately after creation. This rigorous descriptor management prevented descriptor table exhaustion across all eleven concurrent stages.

CONCLUSIONS

This laboratory work provided an in-depth investigation into userland process orchestration, hardware privilege boundary transitions, and pipe-based inter-process communication within the educational xv6 operating system running on the 64-bit RISC-V architecture. By implementing native C utilities directly against xv6 kernel system calls, the assignment verified that the foundational Unix operational model studied in modern Linux environments applies identically to minimalist educational kernels.

The empirical verification of `user/run.c` confirmed that the three-step lifecycle model (`fork`, `exec`, `wait`) functions with complete architectural fidelity in xv6. The child process demonstrated successful binary substitution via `exec()`, confirming that user programs relinquish their memory spaces while the parent reaps the child termination status without resource leakage. Furthermore, analyzing execution failures highlighted key status code conventions, contrasting Linux POSIX status 127 with xv6 status 1.

The development of `user/pingpong.c` demonstrated the necessity of dual unidirectional pipes for deadlock-free bidirectional communication, illustrating how blocking kernel read primitives provide implicit process synchronization without busy-waiting. Finally, the successful implementation of the concurrent prime sieve (`user/primes.c`) demonstrated the power of the Unix pipeline abstraction, proving that complex concurrent processing networks can be constructed reliably within constrained operating system resource boundaries.

REFERENCES

[1] COX, R., KAASHOEK, M. F., MORRIS, R. *xv6: a simple, Unix-like teaching operating system*. Cambridge: MIT CSAIL, 2020. 108 p.

[2] SILBERSCHATZ, A., GALVIN, P. B., GAGNE, G. *Operating System Concepts*. 10th Edition. Hoboken: John Wiley & Sons, 2018. 1040 p. ISBN 978-1-119-32091-3.

[3] PATTERSON, D. A., HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 2nd Edition. Cambridge: Morgan Kaufmann, 2020. 696 p. ISBN 978-0-12-820331-6.

[4] WATERMAN, A., ASANOVIĆ, K. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Version 20211203. RISC-V International, 2021. 142 p.

[5] MCILROY, M. D. A Concurrent Sieve of Eratosthenes. In: *Communications of the ACM*, 1968, vol. 11, no. 8, pp. 560–563.

ANNEX A

Listing 1: Complete implementation of user/run.c

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    if(argc > 1){
      exec(argv[1], &argv[1]);
    } else {
      char *args[] = { "echo", "hello", "from", "the", "child", 0 };
      exec("echo", args);
    }
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Listing 2: Complete implementation of user/pingpong.c

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
```

```
{
  int p2c[2], c2p[2];
  char b;
  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }

  close(p2c[0]); close(c2p[1]);
  write(p2c[1], "x", 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}
```

ANNEX B

Listing 3: Complete implementation of user/primes.c

```c
#include "kernel/types.h"
#include "user/user.h"

void sieve(int left_fd)
{
  int p;
  if(read(left_fd, &p, sizeof(int)) != sizeof(int)){
    close(left_fd);
    exit(0);
  }

  printf("prime %d\n", p);

  int next_pipe[2];
  pipe(next_pipe);

  int pid = fork();
  if(pid < 0){
    fprintf(2, "primes: fork failed\n");
    close(left_fd);
    close(next_pipe[0]);
    close(next_pipe[1]);
    exit(1);
  }

  if(pid == 0){
    close(left_fd);
    close(next_pipe[1]);
    sieve(next_pipe[0]);
    exit(0);
  } else {
    close(next_pipe[0]);
    int n;
    while(read(left_fd, &n, sizeof(int)) == sizeof(int)){
      if(n % p != 0){
        write(next_pipe[1], &n, sizeof(int));
      }
    }
```

```
    close(left_fd);
    close(next_pipe[1]);
    wait(0);
    exit(0);
  }
}

int main(int argc, char *argv[])
{
  int p[2];
  pipe(p);

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    exit(1);
  }

  if(pid == 0){
    close(p[1]);
    sieve(p[0]);
    exit(0);
  } else {
    close(p[0]);
    for(int i = 2; i <= 35; i++){
      write(p[1], &i, sizeof(int));
    }
    close(p[1]);
    wait(0);
    exit(0);
  }
}
```

Listing 4: Makefile modifications registering UPROGS in xv6

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
```

```
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_logstress\
    $U/_forphan\
    $U/_dorphan\
    $U/_sleep\
    $U/_args\
    $U/_mystat\
    $U/_find\
    $U/_run\
    $U/_pingpong\
    $U/_primes\
```